SmartBIS Development Environment Setup v1.2 (As-built)

Headwind MDM과 Android 기기 프로비저닝 환경은 별도 기기 등록·운영 절차서에 기록한다. 해당 문서는 지자체 운영·관리 매뉴얼의 근거이며, 시험 결과에 따라 갱신한다.

- 기준일: 2026-10-02
- 상태: 현재 개발환경 및 확인된 구현을 기술한다. 요구사항·미구현 항목은 System Requirement Specification과 전체 구축 진행현황을 함께 확인한다.
- 이 문서는 이전 구축 이력이나 백업본의 계획 구성이 아니라 현재 프로젝트 파일을 기준으로 한다.

1. 목적과 운영 경계

SmartBIS는 차량에 설치한 Android 앱과 서버 사이에서 콘텐츠·도착정보를 주고받는 시스템이다. Spring Boot는 내장 Tomcat을 포함한 컨테이너로 실행하고, 외부 접근은 Nginx HTTPS 포트 `8443` 하나로 제한한다. 별도 Tomcat 설치, Eclipse 배포, `/sht_webapp/` 경로는 현재 구성에 사용하지 않는다.

관리 대시보드는 별도 공개 서비스가 아니라 같은 HTTPS 진입점 아래의 보호된 운영 UI다. 현재 계정 방식과 운영 기능은 초기 관리자용이며, 다중 사용자·역할 관리까지 구현된 상태는 아니다.

2. 확인된 구성

```
Android Emulator/차량 단말
  └ HTTPS :8443 (에뮬레이터에서는 https://10.0.2.2:8443)
      └ Nginx :443 (사설 CA 인증서 종료, 외부 포트 8443)
          └ Spring Boot 내장 Tomcat :8080 (/api context path)
              ├ PostgreSQL :5432 (Compose 내부망)
              └ content-store 읽기 전용 미디어 제공

내부망 콘텐츠 제공자
  └ upload-store/YYYY-MM-DD/BATCH_ID
      ├ contents.json
      ├ 미디어 파일(선택)
      └ .ready (전송 완료 신호)
          └ Node.js watcher → 검증·SHA-256 확인·DB 트랜잭션·파일 보관
              ├ 성공: content-store/YYYY-MM-DD/BATCH_ID
              ├ 입력 보관: upload-store/archive
              └ 실패 보관: upload-store/error
```

Compose 서비스는 `postgres-db`, `egov-backend`, `node-watcher`, `nginx`다. 호스트로 게시하는 포트는 `8443:443`이며 PostgreSQL `5432`와 Spring Boot `8080`은 Compose 네트워크 내부용이다. DB는 named volume `db-data`를 사용한다. `podman compose down`은 데이터 볼륨을 삭제하지 않지만, `down -v`는 DB 데이터를 제거하므로 임의로 실행하지 않는다.

현재 Compose에는 컨테이너 healthcheck가 없다. `depends_on`은 시작 순서를 정할 뿐 DB/API 준비 완료를 보장하지 않는다. 재기동 직후 `502`가 나면 백엔드 로그와 health endpoint를 확인하고 준비 후 재시험한다. Nginx upstream은

서비스 이름으로 찾으므로 백엔드 재생성 후 일시적 주소 오류가 지속되면 Nginx도 재시작한다.

3. 주요 기술 및 빌드 환경

구성요소	현재 설정	확인 범위
컨테이너	Podman Desktop/Podman Compose	Windows 호스트에서 Compose 프로젝트 실행
데이터베이스	PostgreSQL 15 Alpine	컨테이너 전용 네트워크, named volume
API 서버	Spring Boot 3.3.5, Java 17, 내장 Tomcat	Maven 컨테이너로 패키징, 앱 포트 8080
업로드 감시	Node.js 20 Alpine, PostgreSQL `pg`, Chokidar polling, node-cron	상시 프로세스; 정기 업무는 Asia/Seoul 04:00
TLS 진입점	Nginx Alpine, 사설 CA 발급 서버 인증서	외부 포트 8443 → 컨테이너 443
Android	Kotlin, minSdk 30, targetSdk 37, compileSdk 37, Java 호환 11	Android 11 이상을 대상으로 한 현재 빌드 설정

`podman compose`는 현재 Windows Docker Compose 호환 provider를 호출한다는 안내를 출력할 수 있다. 이는 안내 메시지 자체이며 서비스 기동 성공/실패 판정은 Compose 결과와 컨테이너 상태로 한다.

4. 환경변수와 비밀정보

프로젝트 루트의 `.env`에서 PostgreSQL 접속 및 대시보드 초기 관리자 계정, TAGO 설정 등을 공급한다. `.env`는 Git에서 제외한다. 실제 비밀번호나 서비스키를 문서·소스·명령 결과에 복사하지 않는다. 이미 노출된 비밀정보는 사용 중단·재발급 여부를 검토하고 안전한 저장소로 옮긴다.

Compose에는 `TAGO_ENABLED`, `TAGO_MOCK_MODE`, `TAGO_SERVICE_KEY`, `TAGO_SERVICE_KEY_MODE`, `TAGO_CITY_CODE`, 재시도·간격 설정이 전달되고 Spring 설정에도 바인딩되어 있다. 그러나 설정이 존재하는 것과 API 수집기가 구현된 것은 다르다. 2026-10-02 서버 `.env` 키로 정류소정보 도시코드 API를 직접 호출해 HTTP 200, `resultCode=00`, 장성군 `cityCode=36450`을 확인했다. 버스노선정보 API 활용 신청·승인 후 같은 서버 키로 `BusRouteInfoInqireService/getCtyCodeList`가 HTTP 200, `resultCode=00`을 반환했고, `getRouteNoList?cityCode=36450`도 HTTP 200, `resultCode=00`, 총 105개 노선을 반환했다. 승인 전 `getRouteNoList`의 403은 승인 전 상태의 과거 결과이며 현재 상태가 아니다. 위치조회 예시(`cityCode=25`, `routeId=DJB30300052`)는 HTTP 200, 7건이지만 장성군 실노선 검증과는 별개다. TAGO 수집기는 아직 구현되지 않았고 `TAGO_ENABLED=false`, `TAGO_MOCK_MODE=true` 상태다.

행정안전부 재난문자 외부 수집 환경변수·수집기는 현재 Compose/백엔드에 구성되지 않았다. 재난 표시 엔진 내부 확인은 DB 샘플 자료를 이용한 테스트이며 외부 API 연동과 혼동하지 않는다.

5. 백엔드 실행·경로 설정

`application.properties`의 기준값은 다음과 같다.

```
server.port=8080
server.servlet.context-path=/api
```

```
spring.datasource.url=jdbc:postgresql://postgres-db:5432/${POSTGRES_DB}
content.root=${CONTENT_ROOT:/app/content-store}
server.forward-headers-strategy=framework
```

컨테이너 안의 API 주소는 `/api/...`이며 Nginx는 `/api/` prefix를 유지해 `egov-backend:8080`으로 전달한다. 대시보드 정적 파일은 Spring Boot의 `/api/dashboard/...` 경로로 제공한다. 2026-10-02 Nginx가 외부 `Host` 포트 `8443`을 보존하지 못해 로그인 redirect에서 포트가 빠지던 문제를 `Host $http_host`, `X-Forwarded-Host $http_host`와 Spring forwarded-header 처리로 수정했다. 비로그인 redirect가 `https://localhost:8443/api/dashboard/login.html`로 가는 것까지 확인했으며, 그 뒤 실제 브라우저 로그아웃은 재검증 대상이다.

미디어는 `content-store`를 백엔드에 읽기 전용 bind mount하여 제공한다. 응답은 파일 시스템 리소스로 스트리밍하고 Nginx buffering을 끈다. Node watcher가 DB에 저장하는 `contents.source_path`는 현재 컨테이너 절대경로(예: `/app/content-store/2026-09-25/BATCH_ID/file.mp4`)다. 과거 누적 기록에 상대경로라고 적힌 부분은 정정 이력을 참고한다. 백엔드에서는 경로 정규화 후 허용 root 아래인지 검사해야 한다.

2026-10-04 저장소 위치 정리: 호스트의 영구 미디어 원본 경로는 `server/content-store`이며 `compose.yaml`에서 watcher에 `/app/content-store` 쓰기 가능, backend에 같은 경로 읽기 전용으로 mount한다. 외부 제공 측 수신 경로 `upload-store`는 프로젝트 루트에 그대로 둔다. 컨테이너 경로를 바꾸지 않았으므로 DB의 기존 `source_path` 값과 미디어 API 경로는 유지된다. 서버 이전 시 `server/content-store`와 PostgreSQL 데이터 백업을 함께 옮긴다. 기존 루트 `content-store`는 새 mount와 API 검증이 끝날 때까지 롤백용으로 보존한다.

6. 콘텐츠 인입과 처리 규약

제공 측은 날짜 폴더 아래 배치 폴더를 생성한다. `contents.json`과 해당 미디어 파일을 모두 복사한 뒤 `.ready`를 마지막에 생성한다. `.ready`가 없으면 04:00 테스트를 포함해 처리 대상이 아니다. watcher는 파일 이벤트도 관찰하지만 Windows bind mount에서 이벤트 유실이 있어 Chokidar polling을 사용한다. 파일 안정성은 기본 10초 간격으로 3회 크기·수정시각이 유지되는지 확인한다.

```
upload-store/YYYY-MM-DD/BATCH_ID/
  contents.json       # batch_id와 contents 배열
  poster.jpg          # 필요한 콘텐츠의 첨부 파일
  video.mp4
  .ready              # 모든 전송이 끝난 다음 생성
```

현재 manifest는 한 배치 안의 `contents` 배열을 처리하며 각 콘텐츠에 고유한 `content_id`를 사용한다. DB UPSERT 키는 `content_id`다. 기존 ID로 수정 재전송하면 기존 콘텐츠 행을 갱신하므로 새로운 자료는 새 ID를 사용한다. 다중 파일의 카드뉴스 계약은 아직 확정·구현되지 않았다. 성공 파일은 `content-store/YYYY-MM-DD/BATCH_ID`로 복사되고 입력 배치는 archive로 이동한다. 실패 배치는 이유를 로그로 남기고 `upload-store/error`로 이동한다.

watcher는 배치 실패 시 `content_ingest_batches`에 `FAILED` 상태/시도 횟수/마지막 오류를 기록하고, `content_validation_errors`에 오류 이벤트를 같은 DB transaction으로 추가하도록 구현되었다. 실패 폴더 보관 후 DB 기록을 시도하며 DB 기록도 실패하면 파일은 남고 로그에 기록 실패를 남긴다. 같은 배치 ID를 재전송해 성공 처리하면 batch 상태를 `PROCESSED`로 갱신하고 미해결 오류를 `RESOLVED`로 변경한다. 2026-10-02 실제 PostgreSQL에 의도적인 JSON 오류와 동일 ID 재처리를 수행해 `FAILED`/`OPEN`에서 `PROCESSED`/`RESOLVED`로 바뀌는 것을 검증했고 시험 데이터·폴더를 제거했다. 관리자 브라우저에서 검색 결과가 표시되는지와 DB 장애 fallback은 아직 검증하지 않았

다. 파일 복사와 DB 콘텐츠 transaction은 원자적이지 않으므로 DB rollback만으로 이미 복사한 미디어를 되돌릴 수 없다. 고아 파일 정리 규칙도 운영 시험이 필요하다.

운영 watcher는 상시 실행하며 매일 Asia/Seoul 04:00에 `.ready` 배치를 처리한다. `RUN_PROCESS_NOW=true`는 개발 시 즉시 처리 후 프로세스를 끝내는 일회성 시험 경로다. 테스트 override를 Compose 운영 환경에 상시 넣지 않는다.

7. 데이터베이스 관리

현재 migration 파일 순서는 `V001`부터 `V007`까지다. 두 파일의 `V005` 충돌은 `display_states` 파일을 `V006__display_states.sql`로 변경해 저장소에서 해소했다. 2026-10-02 `V007__arrival_display_state.sql`을 추가해 도착정보 표출 중복 방지 컬럼 2개를 실행 중인 DB에 additive 방식으로 적용하고 백엔드 health `UP`을 확인했다. Flyway/Liquibase 및 migration ledger는 없어 V001~V006의 과거 적용 내역은 앱이 관리하지 않는다. 이후 SQL은 현재 스키마와 적용 여부를 먼저 확인해 명시적으로 적용·기록하고, `down -v`나 DB 초기화는 금지한다.

기존 DB를 유지하며 확인한다. `podman compose down -v`, volume 삭제, DB 재초기화는 데이터 손실 위험이 있어 정상적인 재빌드/재기동 절차로 사용하지 않는다.

8. API 및 관리자 기능

대표 경로(외부 기준, 모두 HTTPS `:8443`):

경로	기능	현 상태
`GET /api/actuator/health`	앱 및 DB health	컨테이너에서 UP 확인 기록
`GET /api/v1/contents?region=...`	날짜·지역 기준 콘텐츠 목록	API 결과 시험됨
`GET /api/v1/media/{contentId}`	파일 미디어 제공	MP4 응답 헤더·파일 크기 확인
`GET /api/v1/arrival?vehicleNumber=...`	최신 TAGO GPS로 다음 정류소/거리 계산, TAGO 노선·정류소 도착예정 API의 초 단위 도착시간을 분으로 반환	응답에 `estimatedArrivalMinutes` 포함(없으면 `null`). TAGO 도착 API는 차량번호를 반환하지 않아 노선·정류소 기준 ETA이며 개별 차량 ETA로 단정하지 않음
`GET /api/v1/display/next?vehicleNumber=...`	도착/재난/콘텐츠 다음 항목 선택	샘플 데이터로 시험; 실측 운행은 미검증
`/api/dashboard/login.html`, `/api/dashboard/index.html`	관리자 로그인/대시보드	포트 포함 로그인 redirect 확인
`GET /api/v1/admin/contents` 및 상태 변경 `PUT`	게시 중지·재개·제외·복구	사용자 시험 확인; 제외는 논리 상태
`/api/v1/admin/display-policies`	표출 정책 조회·수정	화면/API 구현; 실제 운영값의 기관 승인 절차는 미정

경로	기능	현 상태
`/api/v1/admin/ingest-errors...`	DB 오류 이력 검색	watcher 기록 lifecycle 및 인증된 요약/목록 API 응답 확인; 실제 오류 행의 화면 렌더링 대기

대시보드 메뉴가 있다고 해서 모든 운행 지표가 연결되는 것은 아니다. Node watcher 상태는 내부 health API와 관리자 전용 백엔드 프록시를 통해 실제 조회한다. 반면 TAGO 마지막 수집·차량 위치와 Android heartbeat/동기화 이력은 아직 실제 운영 API와 연결되지 않았다.

9. Android 개발·운영 상태

Android 앱은 Kotlin `MainActivity` 기반 기능 시험 구현이다. INTERNET 권한과 앱 내 사설 Root CA 설정이 있고 Emulator는 `10.0.2.2:8443`으로 서버에 접속한다. 현재 차량 번호와 base URL은 소스에 시험값으로 고정되어 있다. 목록/다중 콘텐츠 동기화·운영자용 단말 설정/차량별 배포 설정은 아직 제품 수준이 아니다.

앱은 API 항목을 조회하고 미디어를 앱 cache에 `.tmp`로 내려받은 후 파일명으로 확정한다. 동영상은 Android `VideoView`이며 반복을 끄고 완료 이벤트에서 다음 표출을 요청한다. 이미지와 텍스트도 표시하지만 현재 layout은 전체 화면 1920×1080 운영 UI가 아니라 버튼이 있는 기본 테스트 화면이다. Room, Retrofit/OkHttp, ExoPlayer 및 장기 오프라인 보장 기능은 확인되지 않았다.

최종 운영 화면은 24인치 1920×1080, 큰 글자·고대비·자연친화형 색상, 공지/행사/재난/영상별 전체 화면 템플릿과 동영상 하단의 절제된 반투명 제목을 목표로 협의되었다. 현재 Android 코드에 최종 디자인 및 여러 장 카드뉴스 재생은 구현되지 않았다. 자세한 합의·결정 필요 항목은 UI 협의사항을 기준으로 한다.

10. 보안과 운영 전 점검

- 외부 진입은 Nginx TLS `8443`로 제한하고 내부 서비스 포트는 호스트에 publish하지 않는다.
- 운영용 Root CA 개인키는 서버 컨테이너·APK에 넣지 않는다. Android에는 신뢰할 CA 인증서만 포함한다. 자체 Root CA는 공개 인터넷의 공인 CA와 다르며 단말 배포·폐기·갱신 절차가 필요하다.
- 관리자 인증은 현재 `.env`의 단일 계정에서 출발한다. 다중 계정/권한 분리와 secret manager 적용 전에는 기관 운영용 최종 인증 체계로 간주하지 않는다.
- 서비스키, 계정 비밀번호, 개인키는 로그·문서·Git에 남기지 않는다. 키 입력/출력 명령에 값을 직접 붙이지 않는다.
- PostgreSQL 정기 백업과 실제 복구 시험, DB migration 이력 관리, 업로드 실패 재처리, 로그 보존, 인증서 갱신 및 장애 알림은 운영 이관 전 확정한다.
- Nginx/API 502는 우선 `podman compose ps`, API 컨테이너 로그, Compose 네트워크 서비스명 연결을 점검한다. 고정 컨테이너 IP를 설정값으로 사용하지 않는다.

11. 검증 기준과 남은 작업

사용자가 전달한 성공 결과에는 DB 연결/health, HTTPS CA 검증, 파일 미디어 HTTP 응답, watcher 다중 콘텐츠 적재, 기본 Android 동기화·표출, 관리자 콘텐츠 상태 조작이 포함된다. 2026-10-02 기준으로 로그인 redirect의 `:8443` 보존 및 빌드/컨테이너 반영이 추가 확인되었다. 이 대화 기록에 있는 시험 결과와 현재 파일을 직접 실행해 재검증한 결과는 구분한다.

Node watcher 변경 후 로컬 단위 시험은 다음처럼 실행한다. DB/Podman 없이 순수 오류 분류와 트랜잭션 호출 흐름을 시험한다.

```
cd server/node-watcher
npm test
```

단위 시험만으로 실제 DB 연결을 증명하지는 않는다. 다만 2026-10-02 별도 통합시험에서 실제 PostgreSQL 오류 기록과 재처리 상태 전환을 확인했고, watcher 이미지 빌드 및 컨테이너 재생성도 수행했다. 대시보드 브라우저 행 표시, DB 장애 fallback 및 고아 파일 복구는 아직 남아 있다.

Codex 실행 환경에서 Podman을 쓸 때: 이 Windows 호스트에는 Podman 6.0.2와 실행 중인 WSL 머신이 있으며 설치 파일은 `C:\Users\eun\AppData\Local\Programs\Podman\podman.exe`에 있다. 기본 제한 실행에서는 Windows가 프로세스 시작을 `액세스 거부`로 차단할 수 있고, 승인된 실행에서는 정상 동작한다. 이 경우 PATH를 반복 수정하지 말고 Codex의 해당 실행 권한 요청을 승인한다. 이것은 모든 명령에 대한 샌드박스 제한을 영구 해제한다는 뜻은 아니다.

운영 준비 완료로 간주하기 전에 다음을 구현·시험한다.

1. TAGO 정류소/노선/위치 실제 수집, 인증키 승인, 지역 분류 DB 적재와 실측 100m 판정.
2. 행정안전부 재난문자 외부 API 승인 후 수집기 연결.
3. 관리자 대시보드에서 실제 watcher 실패 이력을 조회하고, DB 장애 fallback·중복/재처리 및 파일 고아 처리 규칙을 시험.
4. JSON·DB 계약을 확정해 포스터+텍스트, 텍스트 전용, 다중 이미지 카드뉴스, 단일 이미지, 동영상 첨부를 표현.
5. 지역 제한 콘텐츠를 `/display/next` 선택 쿼리에도 적용하고 우선순위·반복 정책 통합 시험.
6. Android 24인치 전용 전체화면 디자인, 동영상 완료 후 연속 재생, 카드뉴스 완료 조건, 단말 설정 및 장시간 장애 복구.
7. 운영자 로그아웃 실제 브라우저 확인, 다중 계정 요구 검토, 백업/복구·인증서 만료·장애 알림 시험.

요구사항 ID와 완료 상태는 SRS 및 전체 구축 진행현황에 기록한다. 상세 변경·장애 원인은 기능별 누적 구축 로그를 참조한다.